# TECNOLOGIE E ARCHITETTURE PER LA GESTIONE DEI DATI
# A.A. 2024/2025

## VIRTUALIZZAZIONE E VIRTUALIZZAZIONE LEGGERA



P. ATZENI, L. BELLOMARINI, S. IANNUCCI

# INTRODUZIONE

## Problema iniziale

- IBM vendeva mainframe costosi a grandi organizzazioni
- Necessità di eseguire diversi sistemi operativi sulla stessa macchina
- Applicazioni sviluppate su diversi sistemi operativi

## Soluzione IBM

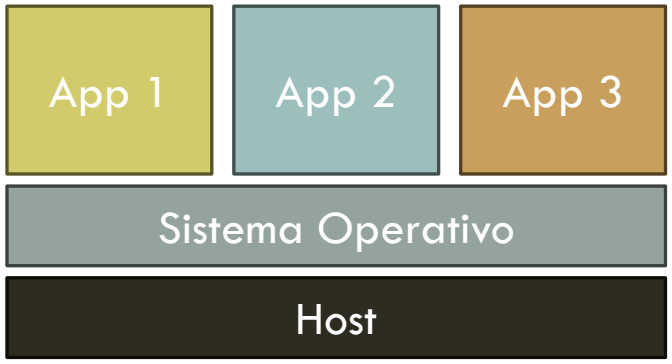
- Introduzione del Monitor di Macchine Virtuali (Virtual Machine Monitor, VMM) o hypervisor
- Il monitor si posiziona tra uno o più sistemi operativi e l'hardware
- Illusione per ogni Sistema Operativo (Operating System, OS) di controllare la macchina.

## Funzionamento del VMM

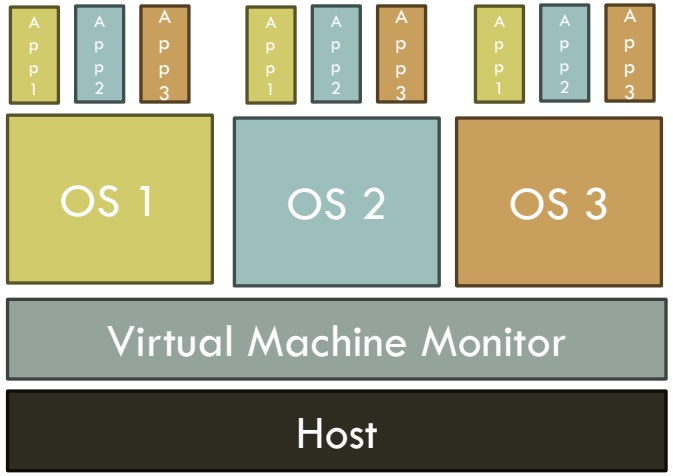
- Il monitor controlla effettivamente l'hardware della macchina fisica
- Multiplexing degli OS sulle risorse fisiche della macchina
- Il VMM funge da sistema operativo per i sistemi operativi, ma a un livello inferiore
- Trasparenza come obiettivo principale

# SISTEMA NON VIRTUALIZZATO VS SISTEMA VIRTUALIZZATO

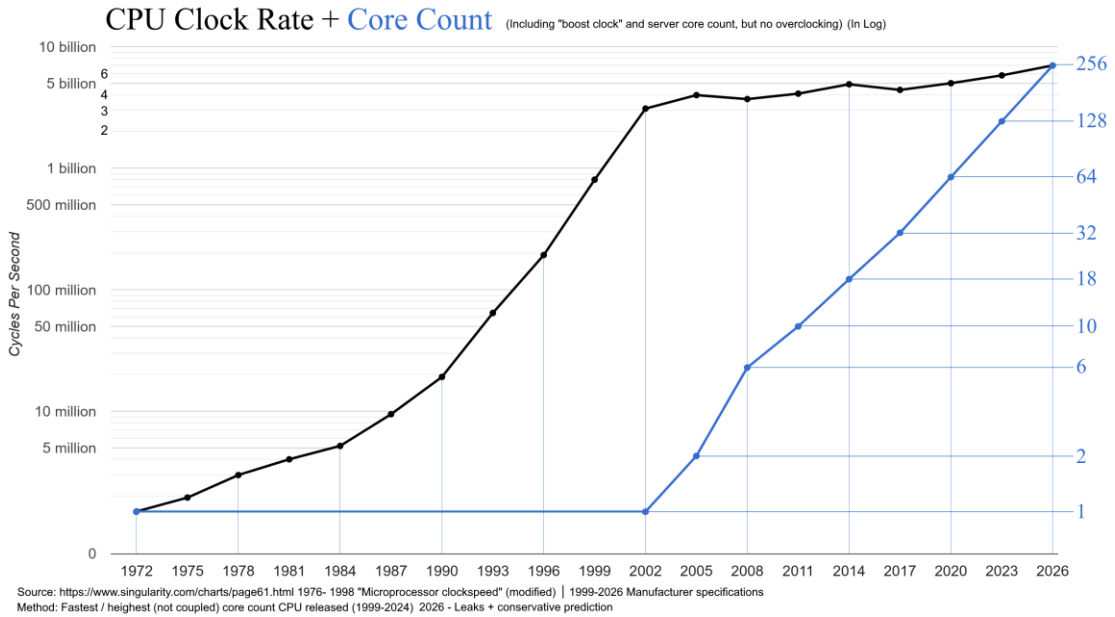
## Sistema non virtualizzato




## Sistema Virtualizzato



# LA POPOLARITÀ DEI VMM




**Motivi della popolarità**

Consolidamento dei server:

- Esecuzione di servizi su macchine diverse con diversi sistemi operativi
- Ogni macchina è poco utilizzata.
- Virtualizzazione consente di consolidare più OS su meno piattaforme hardware
- Riduzione dei costi e semplificazione dell'amministrazione

Virtualizzazione su desktop:

- Esecuzione di un OS differente da quello installato di base
- Accesso ad applicazioni native su un'altra piattaforma

Test e debug

- Sviluppatori scrivono codice su una piattaforma di riferimento
- Necessità di testare il codice su diverse piattaforme
- Virtualizzazione facilita il test su diversi tipi e versioni di OS su una sola macchina

# RICHIAMI DI LIMITED DIRECT EXECUTION

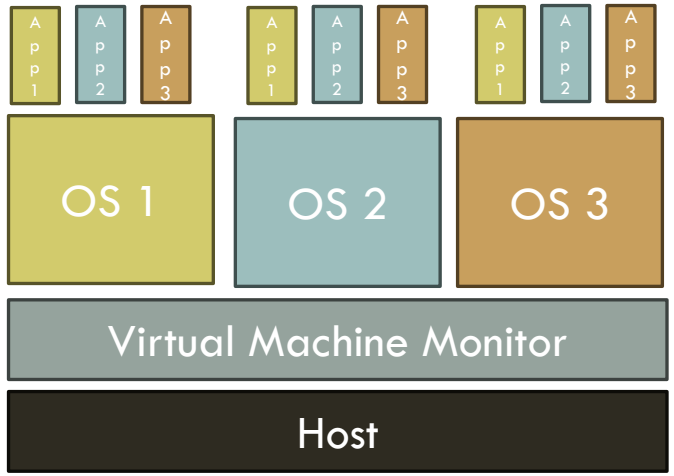
| OS @ boot (kernel mode) | Hardware |
|---|---|
| **initialize trap table** | |
| | remember address of syscall handler |

| OS @ run (kernel mode) | Hardware | Program (user mode) |
|---|---|---|
| Create entry for process list<br>Allocate memory for program<br>Load program into memory<br>Setup user stack with argv<br>Fill kernel stack with reg/PC<br>**return-from-trap** | | |
| | populate regs from kernel stack<br>move to user mode<br>jump to main | |
| | | Run main()<br>…<br>Call system call<br>**trap** into OS |

# RICHIAMI DI LIMITED DIRECT EXECUTION (CONT.)

| OS @ run (kernel mode) | Hardware | Program (user mode) |
|---|---|---|
| | *(Cont.)* | |
| | save regs to kernel stack<br>move to kernel mode<br>jump to trap handler | |
| Handle trap<br>Do work of syscall<br>**return-from-trap** | | |
| | restore regs from kernel stack<br>move to user mode<br>jump to PC after trap | |
| | | …<br>return from main<br>trap (via `exit()`) |
| Free memory of process<br>Remove from process list | | |

# ESECUZIONE DI MACCHINE VIRTUALI SU UN VMM




Invece di utilizzare la *limited direct execution* per fare switch tra processi, la usiamo per fare switch tra VM

Context switch → Machine Switch

- Salvataggio dello stato di una VM (registri, PC, …)
- Ripristino dello stato della VM da eseguire
- Salto al PC della VM da eseguire
- Nota: il PC della VM da eseguire può essere all'interno dell'OS (esecuzione di una chiamata di sistema) o all'interno di un processo in esecuzione sull'OS (applicazione in modalità utente)

# PROBLEMA: ISTRUZIONI PRIVILEGIATE

Il sistema operativo di una VM non può eseguire istruzioni privilegiate, perché queste controllerebbero la macchina fisica invece della macchina virtuale.

Esempio: Aggiornamento del TLB con istruzioni privilegiate

**Soluzione:** Il VMM intercetta i tentativi di eseguire operazioni privilegiate per mantenere il controllo della macchina fisica.

# COME INTERCETTARE LE OPERAZIONI PRIVILEGIATE?

Supponiamo di volere eseguire una *open()*, che è definita come segue:
*open(char *path, int flags, mode t mode)*

Questa si traduce nel seguente codice assembly:

```
open:
    push    dword mode
    push    dword flags
    push    dword path
    mov     eax, 5
    push    eax
    int     80h
```

# COME INTERCETTARE LE OPERAZIONI PRIVILEGIATE? (2)

La trap table è popolata all'avvio del sistema operativo con un'altra operazione privilegiata.

**Soluzione:** il VMM prima intercetta il popolamento della trap table all'avvio del sistema, così da sapere dove si trova questa in memoria. Poi, sfrutta questa informazione per aggiornare il program counter con l'indirizzo opportuno.

# CONFRONTO NELLA GESTIONE DELLE CHIAMATE DI SISTEMA

## Invocazione syscall senza virtualizzazione

| Process | Operating System |
|---|---|
| **1.** System call: Trap to OS | |
| | **2.** OS trap handler: Decode trap and execute appropriate syscall routine; When done: return from trap |
| **3.** Resume execution (@PC after trap) | |

## Invocazione syscall con virtualizzazione

| Process | Operating System | VMM |
|---|---|---|
| **1.** System call: Trap to OS | | |
| | | **2.** Process trapped: Call OS trap handler (at reduced privilege) |
| | **3.** OS trap handler: Decode trap and execute syscall; When done: issue return-from-trap | |
| | | **4.** OS tried return from trap: Do real return from trap |
| **5.** Resume execution (@PC after trap) | | |

# VIRTUALIZZAZIONE LEGGERA

La virtualizzazione, così come l'abbiamo vista finora, risolve i problemi di cui abbiamo parlato, ma ha un costo: **l'overhead.**

Per questo motivo, negli ultimi 10 anni ha preso piede un'altra forma di virtualizzazione: la **virtualizzazione leggera** basata su ***container***

I container sono un metodo di virtualizzazione a livello di sistema operativo che consente di eseguire applicazioni isolate, condividendo lo stesso kernel dell'host.

***Caratteristiche principali:***

- Leggeri: meno overhead rispetto alle macchine virtuali (VM), si avviano in pochi secondi.
- Isolati: usano *namespace* e *cgroup* per separare risorse (CPU, memoria, rete).
- Portabili: eseguibili su qualsiasi sistema con un motore container (e.g., Docker, Podman)
- Efficienti: condividono il kernel dell'host, evitando la duplicazione di SO per ogni istanza.

# ISOLAMENTO NEI CONTAINER

Poiché i container condividono lo stesso kernel, è importante stabilire un meccanismo di isolamento dei processi, così che i processi che appartengono ad un container non abbiano diritti nei confronti dei processi che girano in un altro container.

***Soluzione: namespaces. Diversi tipi:***

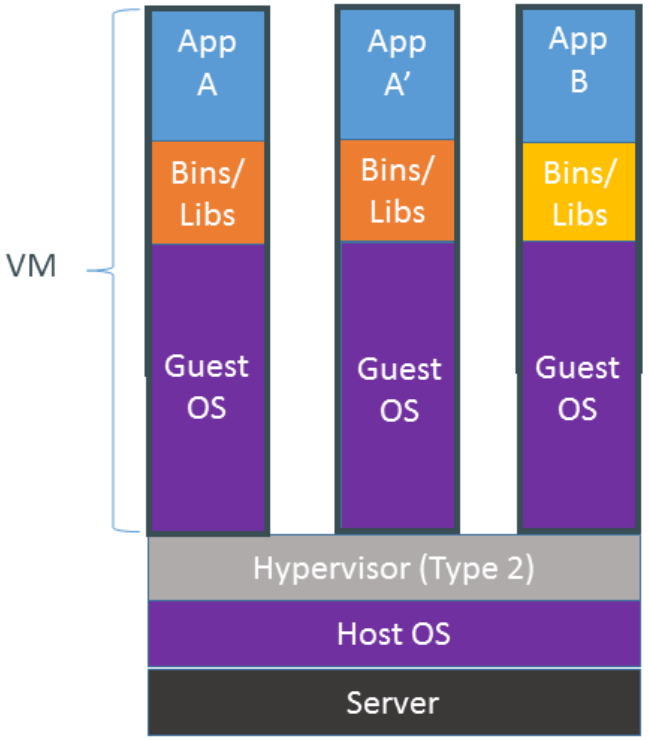
- User namespace: definisce il proprio set di user ID e group ID, permettendo ai processi di avere privilegi root all'interno del namespace senza averli al di fuori.
- PID namespace: Assegna un set indipendente di Process ID (PID). Il primo processo in un namespace ha PID 1, e i suoi figli ricevono numeri PID successivi. Se un processo figlio crea il proprio namespace PID, avrà di nuovo PID 1 al suo interno.
- Network namespace: definisce un network stack indipendente, con propria tabella di routing, indirizzi IP, socket, firewall e altre risorse di rete.
- Mount namespace: Mantiene una lista di mount point indipendenti, permettendo di montare o smontare filesystem senza influenzare l'host.
- IPC namespace: Isola le risorse di Interprocess Communication (IPC), come le POSIX message queue.
- UTS namespace: Permette a un sistema di apparire con nomi host e dominio diversi per processi distinti.

# GESTIONE DELLE RISORSE NEI CONTAINER

I ***control groups (cgroup)*** permettono di limitare, assegnare e monitorare le risorse di sistema per i container.

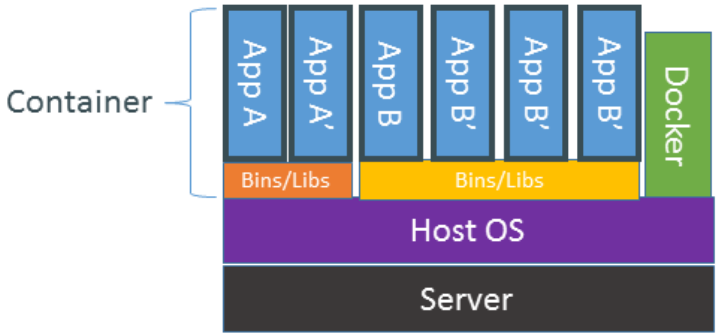
- CPU – Limita il tempo CPU assegnato a un container
- Memoria – Imposta limiti di RAM per evitare che un container consumi tutta la memoria dell'host
- I/O (Block I/O) – Controlla l'accesso ai dischi e velocità di lettura/scrittura
- Rete – Limita la banda disponibile per ogni container
- Device – Restringe l'accesso a dispositivi hardware

# VIRTUAL MACHINES VS CONTAINERS




Containers are isolated, but share OS and, where appropriate, bins/libraries

...result is significantly faster deployment, much less overhead, easier migration, faster restart



# DOCKER

Docker è una piattaforma open-source per lo sviluppo, il test e il deployment di applicazioni in ambienti containerizzati.

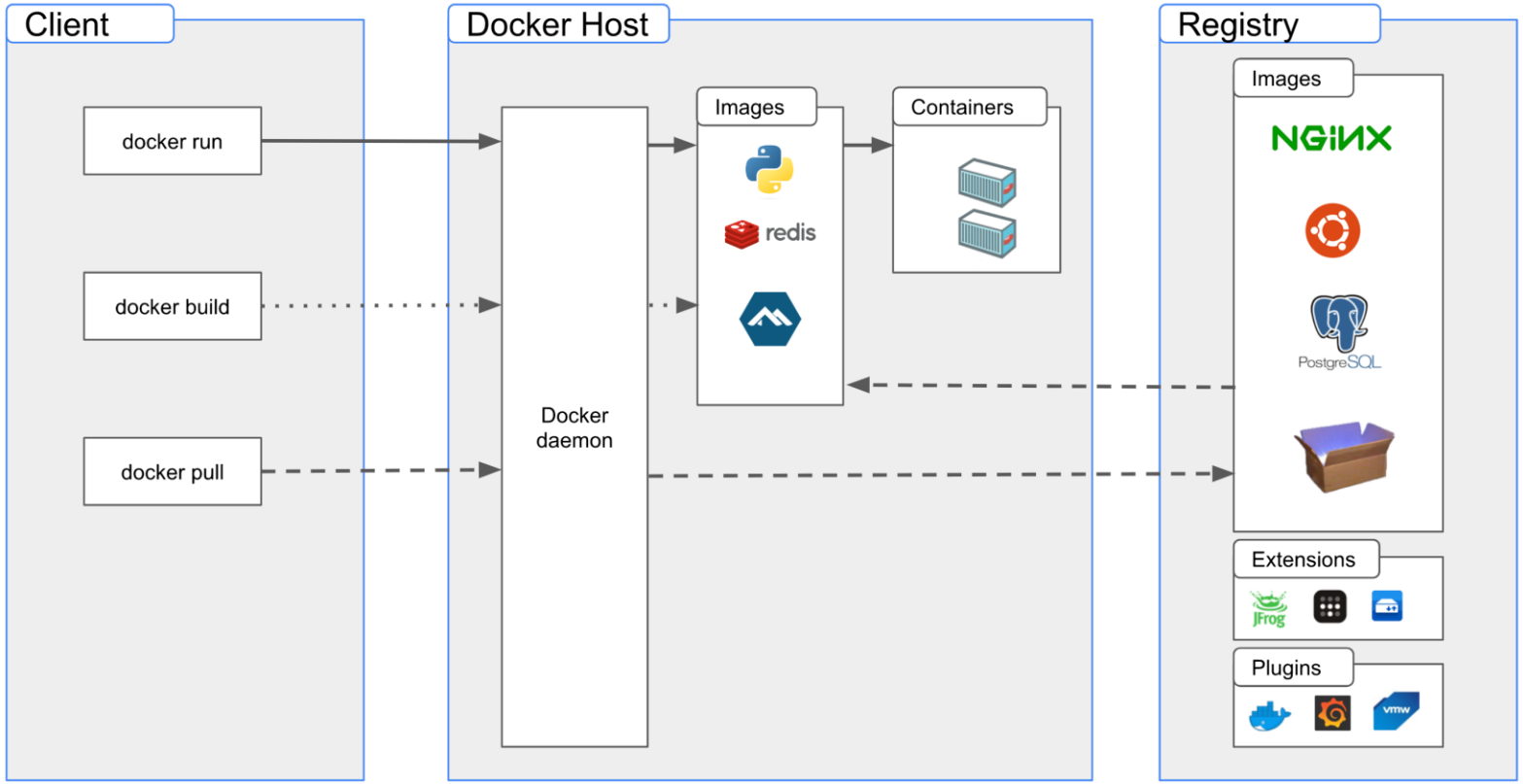
Caratteristiche:

- I container possono essere eseguiti su qualsiasi sistema operativo, garantendo la compatibilità tra ambienti di sviluppo e produzione
- Docker facilita la scalabilità delle applicazioni, permettendo di gestire facilmente picchi di carico

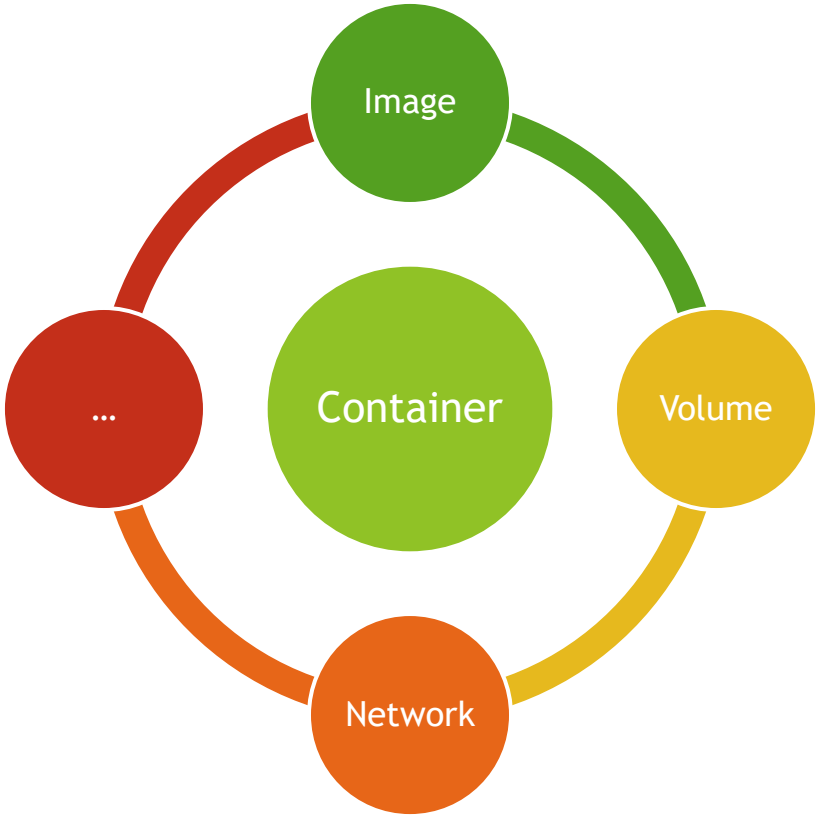
Utilizzi comuni:

- Sviluppo: creazione di ambienti di sviluppo isolati e consistenti
- Test: test delle applicazioni in ambienti controllati
- Deployment: distribuzione delle applicazioni su server di produzione

# L'ARCHITETTURA DI DOCKER



# I COMPONENTI DI DOCKER



# I COMPONENTI DI DOCKER: IMMAGINI

Template *read-only* contenente:

- il root filesystem del sistema che si vuole istanziare
- meta-dati, ad esempio: l'autore dell'immagine, i comandi da eseguire quando il container viene avviato
- le variabili d'ambiente che devono essere impostate
- …

Le immagini sono costituite da *layers*

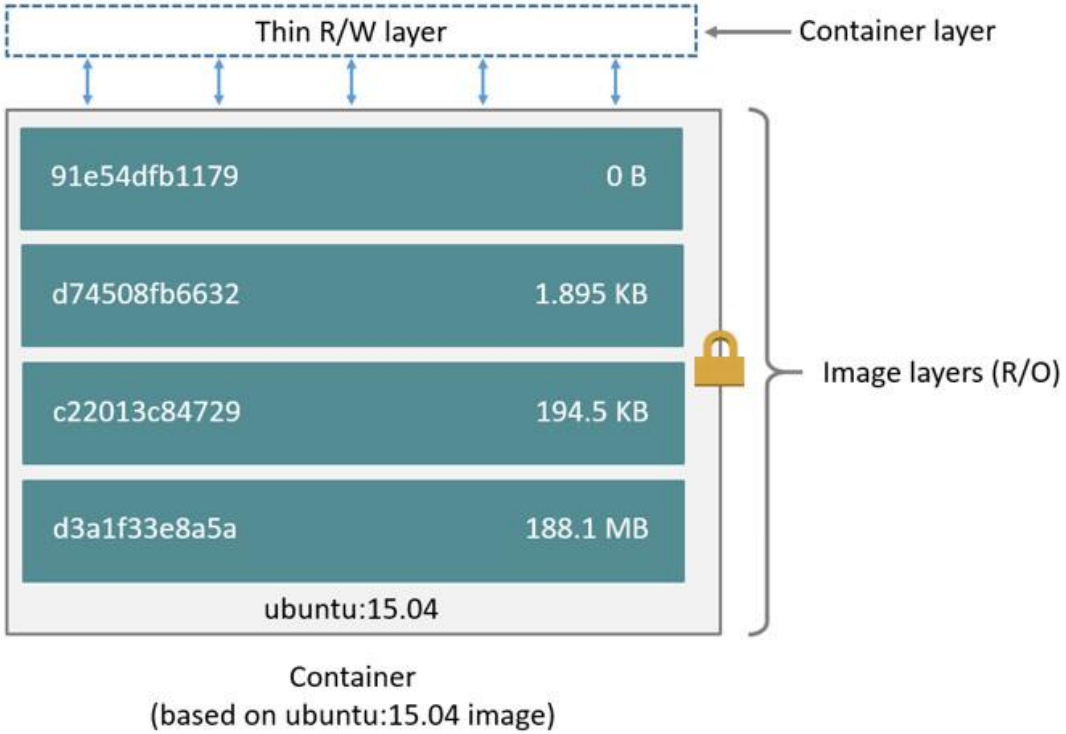
- Ogni layer può aggiungere, cambiare, rimuovere file e/o metadati
- Immagini diverse possono condividere dei layer per ottimizzare l'uso del disco, il tempo di trasferimento, e l'utilizzo di memoria
- Realizzato tramite UnionFS

# ESEMPIO DI IMMAGINE PER UNA WEBAPP JAVA

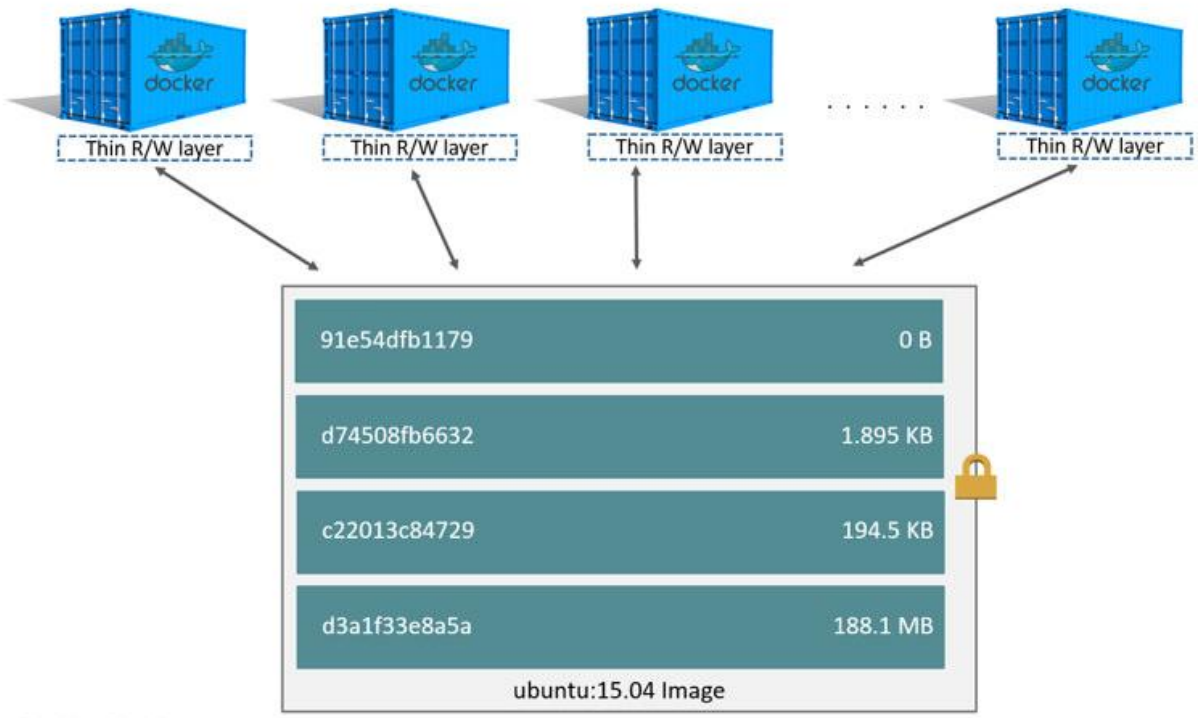
Ognuno dei seguenti elementi corrisponde ad un layer:

- Ubuntu base layer
- Pacchetti e file di configurazione aggiunti dall'Ufficio IT
- JRE
- Tomcat
- Dipendenze dell'applicazione
- Codice dell'applicazione
- Configurazione dell'applicazione

# IL LAYER *READ-WRITE*



# PIÙ CONTAINER POSSONO USARE LA STESSA IMMAGINE



# I COMPONENTI DI DOCKER: VOLUMI

E' buona norma che i container siano *stateless*

- Disaccoppiamento tra sistema, applicazioni, **e dati**
- Il disaccoppiamento tra sistema e applicazioni è realizzato tramite layer dell'immagine
- Il disaccoppiamento rispetto ai dati è realizzato con l'introduzione di un nuovo concetto, quello di ***Volume***

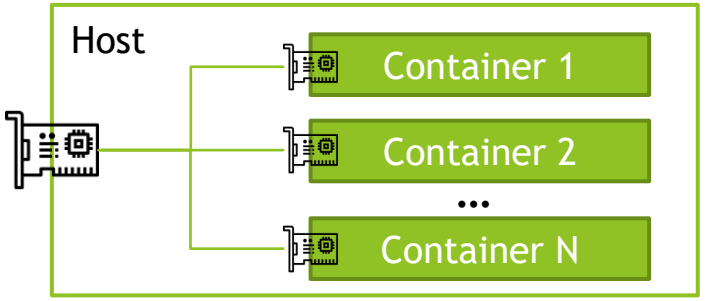
Un volume è una directory che può essere associata ad uno o più container tramite un *mountpoint*

- E' accessibile in lettura e scrittura
- Il suo ciclo di vita è **indipendente** dal ciclo di vita dei container

# I COMPONENTI DI DOCKER: RETI

Anche se ne condividono le risorse, i container lavorano in modo logicamente isolato rispetto alla macchina che li ospita (host) tramite il concetto di namespace

- Un servizio di rete avviato all'interno di un container non è immediatamente visibile sulla rete, a meno di configurazioni aggiuntive
- Necessario che l'host si metta in ascolto su determinate porte e che redirezioni il traffico verso la porta sulla quale il servizio di un dato container è in ascolto



# BUONA NOTIZIA!

La sintassi dei comandi per l'amministrazione dei container Docker è stata aggiornata e "ripulita" nel 2017. Gestire container docker è ora molto semplice, se sappiamo cosa fare.

Comando principale: `docker`

- Per la gestione dei container: `docker container <qualcosa>`
  - Esempio: `docker container ls` mostra l'elenco dei container attivi
- Per la gestione delle immagini: `docker image <qualcosa>`
  - Esempio: `docker image rm <id_immagine>` rimuove l'immagine con ID `id_immagine`
- Per la gestione dei volumi: `docker volume <qualcosa>`
  - Esempio: `docker volume inspect <id_volume>` mostra informazioni dettagliate sul volume con ID `id_volume`
- Per la gestione delle reti: `docker network <qualcosa>`
  - Esempio: `docker network ls` mostra l'elenco dele reti virtuali

# NON CI PIACE IMPARARE A MEMORIA

Con il tempo, imparerete a memoria i comandi principali

- Non è però necessario studiarli tutti subito: l'interfaccia a riga di comando ci aiuta molto.
- Esempio: basta digitare `docker volume` senza ulteriori argomenti e ci verrà restituita una lista di azioni che è possibile eseguire. Stessa cosa vale per `image`, `network`, `container`.

Disponibile una cheat sheet con alcuni dei comandi di utilizzo più frequente:

- https://docs.docker.com/get-started/docker_cheatsheet.pdf